Class-Incremental Learning for Remote Sensing Scene Classification via Stable Diffusion-Based Data Regeneration

Bihui Zhang, Xiangyong Cao, Shihong Wang, and Deyu Meng

***Abstract*—Remote sensing scene classification aims to semantically identify land-cover scenes from satellite or aerial imagery, serving as one of the fundamental tasks in remote sensing interpretation. With the continuous growth in the scale and complexity of remote sensing data, data distributions evolve dynamically, e.g., new categories emerge over time. Thus, continual learning (CL) for remote sensing scene classification has become increasingly important. Among existing CL approaches, experience replay methods achieve the most competitive performance; however, directly applying them to high-resolution and wide-coverage remote sensing imagery incurs substantial storage overhead and raises concerns about data security and privacy. To address these issues, we propose a diffusion-based generative replay scheme that eliminates the need to store real samples while significantly reducing storage requirements. Specifically, we employ low-rank adaptation (LoRA) to fine-tune the stable diffusion (SD) model for each sequential task, enabling it to adapt to evolve data distributions and generate representative samples from prior tasks for efficient replay. Furthermore, to enhance the quality of replayed data, we introduce a data selection mechanism that leverages the previously trained classifier to filter generated samples. Experimental results on multiple remote sensing datasets demonstrate that our proposed method can achieve the superior performance.**

***Index Terms*—Continual learning (CL), remote sensing image scene classification, stable diffusion (SD).**

I. Introduction

REMOTE sensing scene classification aims to semantically identify land-cover scenes from satellite or aerial imagery and assign them to corresponding categories [1]. As one of the fundamental tasks in remote sensing interpretation, it plays a key role in applications, such as land-use analysis [2], geological hazard monitoring [3], and geospatial intelligence [4]. With the increasing diversity of data acquisition methods and improvements in spatiotemporal resolution, remote sensing data are growing in scale, evolving dynamically, and exhibiting multisource heterogeneity. Traditional static learning paradigms generally assume that training data are fixed and fully available, and thus, the trained model can be directly applied for inference [5]. In practice, remote sensing systems typically face open-world scenarios, where previously unseen categories may appear over time [6], [7], [8] and the distribution of remote sensing data changes over time, across different locations and under varying environmental conditions [9]. Moreover, new scenes and categories continually emerge, which makes the trained models forget previous knowledge or even suffer catastrophic forgetting when adapting to new tasks. To address this challenge, continual learning (CL) is a suitable tool that leverages incremental learning mechanisms to enable models to learn new tasks while retaining knowledge from previous tasks. Therefore, developing CL methods that can handle dynamic data distributions and maintain knowledge continuity is of both theoretical and practical significance for remote sensing scene classification. In addition, CL shows unique advantages in applications with limited resources or privacy constraints.

Existing CL methods have developed rapidly, focusing primarily on natural images, with only a few studies addressing remote sensing scene classification [10], [11], [12], [13], [14]. According to [10], these CL methods can be broadly categorized into five types: regularization-based, replay-based, optimization-based, architecture-based, and representation-based approaches. Regularization-based methods [15], [16], [17], [18], [19] retain knowledge from previous tasks by constraining model parameters or intermediate representations during the training of new tasks, such as elastic weight consolidation [15] and distillation-based strategies [20], [21], [22]. Replay-based methods [21], [23], [24], [25], [26], [27] mitigate forgetting by revisiting stored or generated data, thus maintaining performance on old tasks. For instance, Ammour et al. [12] preserved knowledge from previous tasks by storing a task-specific prototype for each task. Optimization-based approaches mitigate interference between tasks by modifying the training procedure, with representative

Received 2 December 2025; revised 11 January 2026; accepted 29 January 2026. Date of publication 6 February 2026; date of current version 6 March 2026. This work was supported in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant JYB2025XDXM101, in part by the National Natural Science Foundation of China under Grant 62272375, and in part by Tianyuan Fund for Mathematics of the National Natural Science Foundation of China under Grant 12426105. *(Corresponding author: Xiangyong Cao.)*

Bihui Zhang is with the School of Mathematics and Statistics, Key Laboratory of Intelligent Networks and Network Security, Ministry of Education, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China.

Xiangyong Cao and Shihong Wang are with the School of Computer Science and Technology, Key Laboratory for Intelligent Networks and Network Security, Ministry of Education, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: caoxiangyong@mail.xjtu.edu.cn).

Deyu Meng is with the School of Mathematics and Statistics, Key Laboratory of Intelligent Networks and Network Security, Ministry of Education, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China, and also with Macao Institute of Systems Engineering, Macau University of Science and Technology, Taipa, Macau.

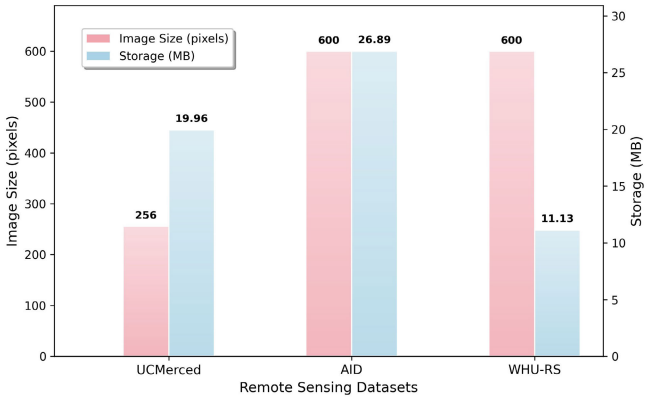
Digital Object Identifier 10.1109/TGRS.2026.3661264

1558-0644 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.



(a)



(b)

Fig. 1. (a) Example images of the BaseballDiamond category from the UC Merced dataset. (b) Image sizes and storage requirements for 100 samples in remote sensing scene classification datasets. Remote sensing images are generally larger than natural images, resulting in higher storage demands.

techniques including gradient projection [28], [29], [30] and metalearning [31], [32]. Architecture-based methods isolate or expand network parameters to prevent new tasks from affecting the existing knowledge [33], [34], [35], [36], [37], [38]. Specifically, Lu et al. [11] proposed task-specific feature extractors, a lightweight feature transfer module, and a dual-constraint loss to enable efficient adaptation to new tasks. Representation-based methods focus on learning robust and generalizable feature to improve cross-task adaptation through pretraining [39], [40], and self-supervised learning [41], [42], [43].

Among these CL methods, replay-based methods have proven highly effective on natural images. However, for high-resolution and wide-coverage remote sensing imagery, storing real samples not only incurs substantial storage overhead, as shown in Fig. 1, but also exposes sensitive geographic information, leading to significant data security and privacy concerns. These issues make experience replay difficult to apply in practical engineering settings or privacy-sensitive scenarios. To tackle this problem, a VAE-based feature replay method [13] has been explored, which replays generated latent features in class-incremental learning for remote sensing scene classification. Nevertheless, this approach may lose some spatial and texture information, as it replays compressed features instead of original images, which may degrade the performance of CL. In contrast to VAE-based feature replay, generative image replay preserves more spatial and texture information.

Therefore, in this work, we propose a generative replay CL method based on stable diffusion (SD). Specifically, we first train a classification model for the current task and then fine-tune the SD according to low-rank adaptation (LoRA) [44] to capture its data distribution. For the next task, data from previous tasks are generated by SD and then integrated with new-task data to update the classifier. The LoRA technique effectively reduces the computational and memory burden of the SD model during CL while alleviating forgetting within the generative model itself. To further improve the quality of generated data, we introduce a data selection mechanism, which uses the classification model trained on previous tasks to filter SD-based generated images. Concretely, the generated images are filtered based on their classification accuracy evaluated by the corresponding classifier.

In summary, the main contributions of our work are threefold.

1) We propose an SD-based generative replay CL method for remote sensing scene classification, which replays remote sensing images by a generative model trained on the datasets from previous tasks. As far as we know, this is the first work of a generative replay CL method for remote sensing scene classification.
2) We introduce the LoRA strategy to effectively fine-tune the SD framework. In addition, we design an efficient data selection mechanism to better select the high-quality generated samples related to the task.
3) Experimental results demonstrate that our proposed method can achieve superior performance. In addition, experimental results show that our method can effectively alleviate catastrophic forgetting while significantly reducing the computational and storage costs compared with conventional replay approaches.

The remainder of this article is organized as follows. Section II reviews the relevant literature. Section III provides the necessary background and introduces the models utilized in our framework. Section IV presents the technical framework and implementation details of the proposed approach. Section V describes the experimental setup, reports the results and discusses the findings in detail. Finally, Section VI concludes this article.

II. Related Works

A. Continual Learning

Existing CL methods are mainly developed in the domain of natural images and can be categorized into regularization-based methods, replay-based methods, optimization-based methods, representation-based methods, and architecture-based methods. Among them, EWC [15] and SI [18] are two representative regularization-based approaches. They estimate the importance of each parameter using the fisher information matrix or its contribution to the total loss and restrict updates to important parameters. LwF [20] is another representative method of function regularization, which mitigates the catastrophic forgetting through knowledge distillation. Replay-based approaches include experience replay and generative replay. Experience replay typically stores a subset of past training samples in a small memory buffer. Given the extremely limited storage capacity, extensive studies such

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.

as [45] and [46] have explored how to construct and utilize this buffer effectively. DGR [24] is a representative method of generative replay, where no raw samples are retained; instead, an additional generative model is trained to capture the distribution of previous tasks and synthesize pseudo-samples for replay during new tasks. Progressive neural networks (PNNs) [33] belongs to structure-based methods, it add a new column for each task while freezing the parameters of previous columns, thereby preventing interference. Optimization-based techniques alleviate forgetting by explicitly manipulating the optimization process. For example, OWM [28] and AOP [29] update parameters in directions orthogonal to the subspace spanned by previous inputs. CaSSLe [41] adopts a representation-based approach, which converts the self-supervised loss to a distillation strategy by mapping the current state of a representation to its previous state. These categories are not mutually exclusive. Many recent studies incorporate multiple strategies to address forgetting. For instance, iCaRL [21] combines knowledge distillation with experience replay and demonstrates strong performance in class-incremental learning. La-MAML [47] integrates the experience replay with metalearning. Among the five major families of CL approaches, regularization-based, architecture-based, and replay-based methods have already been introduced into incremental learning for remote sensing scene classification and have shown promising effectiveness.

B. CL in Remote Sensing Scene Classification

In recent years, several studies have emerged on CL for remote sensing scene classification. Within structure-based approaches, Lu et al. [11] proposed a lightweight incremental learning (LIL) method for remote sensing scene classification, which efficiently adapts to new tasks through task-specific feature extractors and a lightweight feature transfer module, while employing a dual-constraint loss to align data distributions across tasks. Among replay-based approaches, Ammour et al. [12] enhanced the performance of previous land-cover classification tasks by storing one prototype per old task and maximizing the distance between task representations. Subsequently, Ammour [13] alleviated forgetting by learning to regenerate data from prior tasks through analyzing the latent data structure of new tasks. For regularization-based approaches, SCN [48] proposed an asymmetric collaborative network, consisting of a preserving network for long-term memory and a transient network for short-term memory. The method achieves continuous knowledge retention and recall through triple distillation and prior feature fusion. Beyond the aforementioned categories, Bhat et al. [14] employed the curriculum learning to design the learning sequence based on the similarity between new and old classes, enabling a progression from easy to difficult tasks and achieving a better stability–plasticity balance. Among the above approaches, SCN [48] achieves the best performance; however, it requires substantial computational resources during training. In addition, the two methods proposed by Ammour et al. [12] store only a single prototype for each task, which limits their ability to represent the full-data distribution of individual tasks. In [13], the replayed features gradually drift as the number of tasks increases, leading to a degradation in performance.

C. Diffusion Model

In recent years, diffusion models have exhibited remarkable performance in high-fidelity image generation and reconstruction, and have consequently gained increasing attention in the field of remote sensing image analysis. Compared with conventional generative adversarial networks and variational autoencoders, diffusion models reconstruct realistic structures and fine-grained textures from random noise through a gradual reverse denoising process, thereby achieving the superior stability and fidelity. A growing body of research has demonstrated that diffusion-based approaches have already been widely applied in a range of RSI tasks, such as super resolution, cloud removal, change detection, and conditional generation [49], [50], [51], [52], [53]. Moreover, Liu et al. [54] conducted a comprehensive survey on the application of diffusion models in the remote sensing domain, emphasizing their strong generative priors, semantic preservation capabilities, and controllable synthesis mechanisms. Collectively, these advances highlight the vast potential of diffusion models for the generation and reconstruction of RS images. Leveraging their stable sampling mechanism and high-fidelity reconstruction capability, diffusion models are expected to provide more reliable representations of past tasks for generative replay, thereby reducing the risk of catastrophic forgetting. Therefore, in this work, we adopt SD as the foundation generative model for the generative replay.

III. Preliminaries

A. Stable Diffusion

SD is based on the theoretical framework of diffusion models. The core idea is to progressively add noise to a real image in a forward process and then remove the noise step-by-step in a reverse process, recovering a high-fidelity image from random noise. This process enables stable and high-quality image generation.

The forward diffusion process for a real image x_0 is defined as a Markov chain

$$q(x_t|x_0) = \mathcal{N}\left(x_t;\ \sqrt{\alpha_t}x_0, (1-\alpha_t)I\right), \quad t = 1, \ldots, T \tag{1}$$

where α_t is a predefined noise schedule.

The reverse denoising distribution $p_\theta(x_{t-1}|x_t)$ is learned during training, which is equivalent to predicting the noise ϵ. The training loss is formulated as the mean squared error

$$\mathcal{L}_{\mathrm{DM}} = \mathbb{E}_{x_0,\epsilon,t}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2\right]. \tag{2}$$

During inference, starting from random noise $x_T \sim \mathcal{N}(0, I)$, the reverse diffusion process gradually removes noise to generate a high-quality image. This iterative approach ensures stable convergence while preserving fine-grained details, providing significant advantages in generation stability, detail fidelity, and structural consistency.

Traditional diffusion models operate in pixel space, which is computationally expensive for high-resolution images.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.

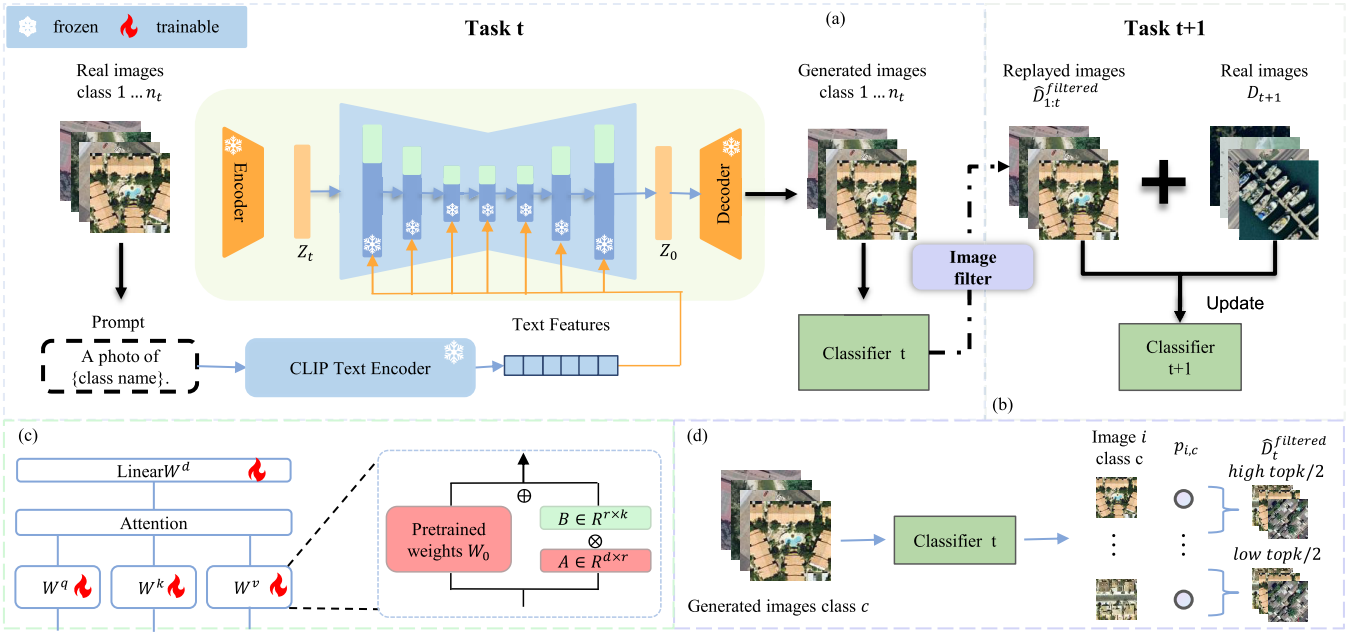


Fig. 2. Overall architecture of our proposed SD-based generative replay method. (a) Image generation for task t. Given class-specific text prompts, a task-adapted SD model for task t generates synthetic images. For each class of task t, class-specific text prompts are constructed based on the class labels in the form of "a photograph of {class name}." (b) Training classifier for task $t+1$. The classifier is updated when the new task $t+1$ arrives, using the generated data of the previous t tasks and the real data of task $t+1$. Note that the generated data are filtered by the classifier t. (c) LoRA in SD. LoRA modules are inserted into the attention layers of SD, where the pretrained weights are frozen and only the lightweight LoRA parameters are optimized for each task. (d) Image filter. Generated images are evaluated by the classifier from task t, and a two tail top-k selection strategy based on the prediction confidence is applied to retain high-quality and diverse replay samples.

SD introduces the latent diffusion mechanism, in which a variational autoencoder first maps images to a lower dimensional latent space, and diffusion is applied in this space to reduce the computational cost. The VAE encoder maps a high-resolution image to a latent vector

$$z_0 = \mathcal{E}(x_0). \tag{3}$$

The forward diffusion in latent space is defined as follows:

$$q(z_t|z_0) = \mathcal{N}\left(z_t;\ \sqrt{\alpha_t}z_0, (1-\alpha_t)I\right). \tag{4}$$

The reverse denoising is performed by a U-Net $\epsilon_\theta(z_t, t, c)$, where c is a text condition vector generated by a CLIP text encoder. After the reverse diffusion, the latent vector is decoded back to an image via the VAE decoder

$$\hat{x} = \mathcal{D}(z_0). \tag{5}$$

Currently, there exist general-purpose SD models pretrained on the LAION dataset [55]. However, to adapt these models to specific tasks or style-constrained applications, fine-tuning is required. Directly fine-tuning the entire model is computationally expensive and prone to overfitting. Therefore, parameter-efficient fine-tuning strategies, such as LoRA [44], are commonly employed to allow the rapid adaptation of the model to new tasks or specific datasets under limited resources, while preserving its original generative capabilities.

IV. Methods

In a class-incremental learning scenario for remote sensing scene classification, the model is required to sequentially learn tasks containing training data with different distributions, while maintaining a good performance on the test samples of all previously learned tasks. Specifically, the dataset S does not arrive in its entirety at once, but rather as multiple data streams, corresponding to multiple learning tasks. Suppose there are T tasks, and each task $t \in \{1, 2, \ldots, T\}$ is associated with a training set $\mathcal{D}_t^{\text{train}} = \{X_t, Y_t\}$ and a testing set $\mathcal{D}_t^{\text{test}}$, where X_t denotes the training instances and Y_t denotes the corresponding class labels. Furthermore, for any two tasks $i, j \in \{1, 2, \ldots, T\}$, their class label sets are disjoint, i.e., $Y_i \cap Y_j = \emptyset$, for $i \neq j$. The model f_θ is required to learn the T tasks sequentially. After completing training on the dataset $\mathcal{D}_t^{\text{train}}$ of the tth task, it must be validated on the testing sets of all tasks learned so far, with the expectation that the model performs well across all these testing sets. Formally, let the model parameters after training on task t be denoted by θ_t. The model learns a mapping function $f_{\theta_t} : \hat{y} = f_{\theta_t}(x)$, where x represents the input remote sensing images whose task identity is unknown. The objective of the model is to correctly predict the label of any input x from previously learned tasks.

A. Overview

To address the problem of catastrophic forgetting of previously learned knowledge while alleviating the excessive memory consumption caused by storing past data, we design a generation-based replay framework built upon SD to achieve the efficient knowledge retention and transfer. As illustrated in Fig. 2, the overall framework consists of two primary modules: a classification model and an SD-based generative

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.

model. After completing the training of task t, our method fine-tunes the generative model G_t using LoRA so that it can learn and simulate the data distribution of the current task. When training task $t+1$, we first use the generative models $\{G_1, G_2, \ldots, G_t\}$ fine-tuned for each previous task to synthesize representative samples of the old tasks. To better utilize the generated data, we introduce a data filtering mechanism based on the classifier's confidence evaluation. Specifically, the classifier trained on previous tasks is used to assess the confidence of each generated sample, and only those with higher classification accuracy are retained as replay data. This ensures the semantic consistency and visual fidelity of the generated samples used for replay.

The proposed framework effectively mitigates the catastrophic forgetting without relying on the storage of historical data. On one hand, the LoRA-based SD fine-tuning mechanism enables the generator to rapidly adapt to new tasks while preserving its ability to recall knowledge from the previous ones. On the other hand, the data filtering strategy guarantee the quality of the generated data, thereby promoting stable updates of the classifier. Overall, this framework not only significantly reduces storage costs but also effectively alleviates catastrophic forgetting in class-incremental learning, providing an efficient solution for CL in remote sensing scene classification. The specific workflow is illustrated in Algorithm 1.

Algorithm 1 Proposed SD-Based Generative Replay Framework

Require: T tasks with dataset streams $\{\mathcal{D}_1, \ldots, \mathcal{D}_T\}$; classifier f_θ; generator G is composed of the frozen backbone weights along with the LoRA modules Δ

for $t = 1$ to T **do**
  if $t = 1$ **then**
    Train classifier f_{θ_1} on dataset $\mathcal{D}_1$
  else if $t > 1$ **then**
    Update classifier f_{θ_t} on real data $\mathcal{D}_t$ and generated data $\{\hat{\mathcal{D}}_1^{\text{filtered}}, \ldots, \hat{\mathcal{D}}_{t-1}^{\text{filtered}}\}$
  end if
  Train Generator G_t with dataset $\mathcal{D}_t$ to adapt to task t.
  Generate previous-task data $\{\tilde{\mathcal{D}}_{t-1}, \tilde{\mathcal{D}}_{t-2}, \ldots, \tilde{\mathcal{D}}_1\}$
  Select data $\hat{\mathcal{D}}_t^{\text{filtered}}$ for each category, from $\{\tilde{\mathcal{D}}_{t-1}, \tilde{\mathcal{D}}_{t-2}, \ldots, \tilde{\mathcal{D}}_1\}$ by confidence scores.
end for

Ensure: Classifier f_{θ_T}

B. Task-Specific LoRA Adaptation in SD

To reduce the computational cost of model fine-tuning, we do not update the entire SD model with full parameters. Instead, we adopt the LoRA technique [44] to perform lightweight task-specific adaptation, enabling the generative model to gradually accommodate an expanding distribution of data categories. Specifically, LoRA injects trainable low-rank matrices into each layer of a pretrained model while keeping the original weights frozen, thereby enabling the efficient fine-tuning. During training, only these low-rank matrices are optimized, which significantly reduces the number of trainable parameters and memory consumption while preserving the knowledge learned by the pretrained model. Let the pretrained weight matrix of a given layer be $W_0 \in \mathbb{R}^{d\times k}$, and LoRA introduces two low-rank matrices $A \in \mathbb{R}^{d\times r}$, $B \in \mathbb{R}^{r\times k}$, and $r \ll \min(d,k)$. The effective weight of the layer during fine-tuning is then given by

$$W = W_0 + \Delta W = W_0 + BA \tag{6}$$

where only A and B are trainable while W_0 remains frozen. This formulation reduces the number of trainable parameters from $d \times k$ to $r \times (d+k)$, enabling the model to adapt to task-specific patterns. During the forward pass, the output of an LoRA-injected layer is computed as follows:

$$y = (W_0 + BA)x \tag{7}$$

where x denotes the input feature map and y denotes the updated output. In SD, LoRA is typically integrated into the convolutional layers of the U-Net and the attention modules. This approach allows the model to efficiently capture the domain-specific information while preserving the original generative capacity of the pretrained SD and semantic consistency.

In addition, unlike conventional generative replay methods, our approach does not iteratively update the same set of LoRA parameters at each stage of the task. For each new task, an independent LoRA parameter module is introduced, allowing the model to progressively learn task-specific distributional characteristics while keeping the backbone network frozen. This design allows the generator to maintain its capability for previous tasks, avoids cross-task interference due to parameter sharing, and provides a stable solution for multitask incremental generation.

Specifically, when a new task $\mathcal{D}_t$ arrives, LoRA modules are injected into attention and feed-forward layers of the SD model's U-Net, creating a new set of LoRA parameters $\{\Delta W_t^{(l)}\}$. Let the original weight matrix of the lth layer be $W_0^{(l)} \in \mathbb{R}^{d\times k}$. Through LoRA, this layer is represented as follows:

$$W^{(l)} = W_0^{(l)} + \Delta W_t^{(l)} = W_0^{(l)} + B_t^{(l)} A_t^{(l)} \tag{8}$$

where $A_t^{(l)} \in \mathbb{R}^{d\times r}$, $B_t^{(l)} \in \mathbb{R}^{r\times k}$, and $r \ll \min(d,k)$. During fine-tuning, only the low-rank matrices $A_t^{(l)}$ and $B_t^{(l)}$ are updated using the current task data $\mathcal{D}_t$, while the backbone weights $W_0^{(l)}$ and the LoRA parameters of previous tasks $\{\Delta W_1^{(l)}, \ldots, \Delta W_{t-1}^{(l)}\}$ remain frozen. After training on task t, the LoRA module for that task is independently stored for the subsequent generative replay. In summary, each task's generative capacity is captured by an independent LoRA layer, which effectively avoids gradient conflicts and parameter interference across tasks. Moreover, since these LoRA parameters account for only a small portion of the total model, the approach is both computationally and memory efficient.

C. Image Generation

We employ an SD that is independently fine-tuned with LoRA for each task, as described in Section IV-B, to generate samples for every class of the previous tasks. During the sample synthesis phase, textual prompts are constructed based on the category labels of the old dataset, allowing the diffusion

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.

model to produce images that are semantically consistent with each category. For each category of task i, a prompt is automatically generated based on the category label, explicitly guiding the model to focus on the visual features of the corresponding category.

Specifically, when generating data for the first $t-1$ tasks during task t, for the ith task ($1 \le i \le t-1$), the base SD model together with the task-specific LoRA parameters trained for task i are used to generate samples for each category within that task. For each category of task i, a prompt in the form of "a photograph of {class name}" is automatically generated based on the category label, explicitly guiding the model to focus on the visual features of the corresponding category. Then, the textual prompts are feed into a pretrained CLIP text encoder to obtain the corresponding text feature vectors, which are used to guide the conditional generation of SD. Let the textual prompt for class j be text_j and be the CLIP text encoder as $E_{\text{txt}}(\cdot)$. The text feature can then be represented as follows:

$$z_j^{\text{text}} = E_{\text{txt}}(\text{text}_j) \in \mathbb{R}^d \tag{9}$$

where d denotes the dimension of the CLIP text feature. In each step of the diffusion process, the U-Net of the generative model takes the text feature z_j^{text} as a conditional input, which is fused with the image latent variable x_t through a cross-attention module, ensuring that the generated samples are semantically consistent with the textual description. Specifically, at the tth step of the diffusion model, the conditional noise prediction can be expressed as follows:

$$\hat{\epsilon}_t = f_\theta\left(x_t, t, z_j^{\text{text}}\right) \tag{10}$$

where f_θ denotes the U-Net network with conditional input, x_t is the latent variable at step t, and $\hat{\epsilon}_t$ represents the noise predicted by the model. This class-conditional generation approach enables the model to better preserve category-specific features during synthesis, thereby facilitating controlled reconstruction of the data distribution of prior tasks.

After obtaining the class-specific text features, the system loads the frozen backbone weights along with the historical LoRA modules $\{\Delta_1, \ldots, \Delta_{T-1}\}$ and selectively activates the corresponding task-specific LoRA parameters. For instance, activating Δ_i generates samples $\hat{\mathcal{D}}_i$ consistent with the data distribution of task i. We then use this model to generate samples for all classes of the current task i. Furthermore, to mitigate the class imbalance caused by the limited number of generated images, the equal number of synthetic samples is generated for each old category to match its real image count. In this way, the constructed joint training set for the current task maintains a balanced class distribution, which helps the classifier remain stable during incremental learning. By preserving per-class sample balance, the model can perform more stable optimization and updating during the incremental learning process, effectively reducing potential bias toward newly introduced categories.

D. Image Filter

To improve the quality of generative replay samples and maintain their semantic consistency, we evaluate and filter the generated samples using the classifier trained during the current incremental learning process. Specifically, for the generated data corresponding to the first t tasks, the classifier that has been continually updated on these tasks is employed to assess the quality of the generated images. This classifier has effectively undergone CL on prior task data through the generative replay and can therefore provide reliable judgments on the semantic fidelity of the generated samples from previous tasks. For each generated sample, the classifier outputs the predicted probabilities for all categories, and these samples are then ranked according to the predicted probability of the target category. In general, only the top-k of samples are retained, removing low confidence or semantically ambiguous samples to ensure the semantic quality and reliability of the generative replay set. However, we observe that images generated by SD typically receive high-confidence scores from the classifier, indicating that their semantic consistency is already reasonably ensured. In this case, directly selecting the top-k highest confidence samples may result in highly similar samples, which fail to fully reflect the diversity of the distribution learned by the generative model. Therefore, we introduce a two-sided sampling strategy to enhance the sample diversity. Specifically, we first rank all generated samples according to the classifier's predicted probability for the target class, and then select $k/2$ samples from the high-confidence end and $k/2$ samples from the low-confidence end. In this way, the selected set includes both highly reliable samples with clear semantics and more diverse samples near the decision boundary, thereby enhancing the diversity of the replay data while maintaining the semantic reliability.

Formally, let $\hat{\mathbf{x}}_i^{(c)}$ be the ith generated sample of class c, and let $f_{\theta_t}(\cdot)$ be the classifier trained up to task t. The classifier outputs the probability distribution

$$\mathbf{p}_i = f_{\theta_t}\left(\hat{\mathbf{x}}_i^{(c)}\right) = \{p_{i,1}, \ldots, p_{i,C}\}, \quad \sum_{j=1}^{C} p_{i,j} = 1 \tag{11}$$

where $p_{i,j}$ represents the predicted probability that the generated sample i belongs to its target class j. After ranking samples by $p_{i,c}$, the filtered set is constructed as follows:

$$\hat{\mathcal{D}}_c^{\text{filtered}} = \text{Top}-k_{\text{high}}(p_{i,c},\, k/2) \;\cup\; \text{Top}-k_{\text{low}}(p_{i,c},\, k/2) \tag{12}$$

where $\text{Top}-k_{\text{high}}$ selects samples from the high-confidence end and $\text{Top}-k_{\text{low}}$ selects samples from the low-confidence end.

Finally, the replay dataset for classifier training is given by

$$\hat{\mathcal{D}}_t^{\text{filtered}} = \bigcup_{c=1}^{C_t} \hat{\mathcal{D}}_c^{\text{filtered}}. \tag{13}$$

E. Training Classifier

During classifier training, we first use the SD to generate data for all previously seen tasks, $\hat{\mathcal{D}}_1, \hat{\mathcal{D}}_2, \ldots, \hat{\mathcal{D}}_t$, and then filter the generated data following the procedure described in the previous section. The classification model is ultimately trained using the filtered data, $\hat{\mathcal{D}}_{1:t}^{\text{filtered}}$, together with the current

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.

task data $\mathcal{D}_t$. These generated samples are then combined with the current task data $\mathcal{D}_{t+1}$ to form the joint training set

$$\tilde{\mathcal{D}}_{t+1} = \mathcal{D}_{t+1} \cup \hat{\mathcal{D}}_{1:t}^{\text{filtered}} \tag{14}$$

which is used to update the classification model. The loss function is defined as follows:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\tilde{\mathcal{D}}_{t+1}|} \sum_{(x,y)\in\tilde{\mathcal{D}}_{t+1}} \sum_{c=1}^{C} \mathbf{1}_{[y=c]} \log \hat{p}_c(x). \tag{15}$$

V. Experiments and Results

A. Experimental Setting

We begin by evaluating our model's performance against several classical methods in class-incremental scenarios, utilizing three benchmark datasets for RSI scene classification. Then, we compare the performance of the proposed methods under different backbone architectures and analyze the impact of image filtering on performance. Finally, we visualize the images generated by SD model and demonstrated the increase in storage space occupied by the model as the number of tasks grew.

1) Datasets: We selected three remote sensing datasets for our experiments. The UC Merced [56], AID [57], and WHU-RS19 [58] datasets were utilized for the comparison of algorithms. Specifically, the UC Merced dataset comprises 21 classes, with 100 images per class. The AID dataset consists of 30 classes, each having 220–420 images. The WHU-RS19 dataset includes 19 classes, each containing approximately 50 samples. For the UC Merced and AID datasets, they were equally partitioned into three and six tasks, respectively. As for WHU-RS19, it was organized into four tasks: the first with four classes and the rest with five classes each.

2) Evaluation Metrics: Referring to [10], we utilize average accuracy (AA), forgetting measure (FM) and intransience measure (IM) as evaluation metrics.

1) *AA:* The average classification accuracy of the model across all learned tasks (from task 1 to task k) after completing the kth task

$$\text{AA}_k = \frac{1}{k}\sum_{j=1}^{k} a_{k,j} \tag{16}$$

where $a_{k,j}$ denotes the model's accuracy on the testing set of task j after learning task k. This metric measures the model's current knowledge retention. It directly reflects the model's average mastery of all previous tasks after undergoing multiple rounds of incremental learning. A higher AA value indicates better control over model forgetting and more stable overall performance.

2) *FM:* FM directly measures the model's degree of forgetting for individual old tasks. It is quantified as the difference between the peak performance achieved by the task at any previous point and its performance at the current evaluation

$$f_{j,k} = \max_{i\in\{1,\ldots,k-1\}} (a_{i,j} - a_{k,j}) \quad \forall j < k. \tag{17}$$

At the kth task, the FM is defined as the average forgetting across all previously learned tasks

$$\text{FM}_k = \frac{1}{k-1}\sum_{j=1}^{k-1} f_{j,k}. \tag{18}$$

3) *IM:* The IM metric evaluates a model's ability to acquire new tasks in CL and is defined by the performance gap between the model and an ideal reference model on the given task. Specifically, it is defined as the discrepancy between the performance of the model under the CL setting and that of an ideal reference model (e.g., a jointly trained model). A larger IM value indicates a weaker ability to learn new tasks, as the CL process incurs a more severe performance degradation

$$\text{IM}_k = a_k^* - a_{k,k} \tag{19}$$

where a_{k^*} denotes the ideal performance on the kth task, obtained by a reference model trained jointly on all tasks up to k.

3) Training Details: We employ ResNet-34 as the classifier in our experiments, and the SD v1.4 as the base generator, which is not further pretrained on remote sensing images. Except for continuous learning methods based on specific architectural designs, all classical continuous learning approaches compared in this article utilize this architecture as this baseline. The datasets are divided into training, validation, and testing sets in an 8:1:1 ratio, with all images resized to 256 $\times$ 256 pixels. Hyperparameter settings for the comparison methods are as follows. The parameters λ and α are set to 1 and 0.5 in EWC, respectively, where λ controls the relative importance of the old task compared to the new one, and α determines the fusion ratio of the old and new fisher information. In the LwF method, we set the loss balance weight λ to 1 and the distillation temperature T to 2. The parameter configurations of r-walk [19] are consistent with those used in the EWC method. For all comparison methods, the SGD optimizer is adopted with a learning rate of 0.001. Training was conducted for 100 epochs. In our experiments, we employed the adam optimizer with a learning rate of 0.00001. Training was conducted for 50 epochs. For our method, the sampling step size was set to 30, and the conditional guidance parameter guidance_scale was set to 7.5. Experiments are conducted on NVIDIA GeForce RTX 4090, with a batch size of 4.

B. Comparison and Analysis

1) Comparison With Class-Incremental CL Methods: To validate the effectiveness of our proposed method, we conducted a systematic comparison with multiple classical class-incremental CL methods, as well as methods specifically designed for remote sensing scene classification tasks. The methods included in the comparison are EWC [15], LwF [20], SI [18], r-walk [19], SCN [48], and Nammour [12]. Among these, EWC [15] and SI [18] serve as key representatives for approaches based on weight regularization. They measure parameter importance in different ways: EWC computes the fisher information matrix, while SI assesses the impact of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.

TABLE I

COMPARISON OF OUR METHOD WITH OTHERS USING RESNET-34 AS THE BACKBONE. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD, AND THE SECOND-BEST RESULTS ARE INDICATED WITH AN UNDERLINE

Method	UCMERCED			AID			WHU-RS		
	AA↑	FM↓	IM↓	AA↑	FM↓	IM↓	AA↑	FM↓	IM↓
Finetuning	0.8857	0.1429	-0.0191	0.5804	0.3404	0.1048	0.6059	0.4918	-0.0282
Ewc [15]	0.8667	0.1500	-0.0048	0.5801	0.3495	0.0974	0.6569	0.4459	-0.0449
Lwf [20]	0.8714	0.1571	-0.0143	0.6119	0.3265	0.2242	0.5871	0.5056	-0.0199
SI [18]	0.8857	**0.1357**	-0.0143	0.5804	0.3404	0.1048	0.6059	0.4918	-0.0282
r_walk [19]	0.8619	0.1571	-0.0048	0.5783	0.3565	0.0934	0.6569	0.4459	-0.0449
SCN [48]	0.5286	0.5571	0.0619	0.3564	0.5284	0.1721	0.2891	0.5585	0.2385
Nammour [12]	0.4667	0.2857	0.3048	0.3188	0.4629	0.2891	0.3271	0.5370	0.2718
ours	**0.8953**	0.1571	**-0.1143**	**0.7225**	**0.3199**	**-0.0202**	**0.8447**	**0.2070**	**-0.2141**

parameter changes on the loss. r-walk [19] integrates the ideas of EWC and SI, introducing a regularization path constraint in the parameter space. LwF [20], a representative of function-regularization-based methods, employs a knowledge distillation mechanism. During the learning of new tasks, it uses the outputs of the previous model as soft targets to retain knowledge from prior tasks. Nammour and SCN are two methods specifically proposed for CL in remote sensing scene classification. Nammour [12] constructs task-level prototypes for each historical task and maximizes the distance between different tasks in the feature space to address forgetting. SCN [48] employs an asymmetric collaborative memory update mechanism. It alternately integrates long-term and short-term memories to enhance the model's CL capability.

Quantitative Analysis: Table I presents the experimental results of our method in comparison with the aforementioned approaches under various task settings. In the experiments, we reported the AA across all historical task test sets after completing the final task, as well as the degree of FM. From the results, it can be seen that our method consistently demonstrates a stable advantage across the three remote sensing scene classification datasets. First, our method surpasses finetuning in all three metrics across the datasets. For example, on the WHU-RS19 dataset, our method achieves an AA of 0.8447, representing an improvement of approximately 39% over finetuning, with FM decreasing from 0.4918 to 0.2070 and IM decreasing from −0.0282 to −0.2141. This indicates that our method effectively mitigates forgetting, and while emphasizing the preservation of historical knowledge, it does not hinder the learning of new tasks; instead, it can even facilitate the accumulation of knowledge for new tasks to some extent. Second, our method shows significant improvement over all other classical CL approaches. On the UCMerced dataset, our method achieves an AA of 0.8953, exceeding EWC, SI, LwF, and r-Walk by approximately 1%–3%. The FM value is 0.1571, slightly higher than that of EWC and SI, while IM decreases to −0.1143. Moreover, parameter regularization-based methods such as EWC, SI, r-Walk, and the distillation-based LwF perform significantly worse than our method on the AID and WHU-RS19 datasets. These methods rely solely on constraining parameter changes, which makes them unable to cope with distribution shifts and conflicts between old and new knowledge during CL, resulting in a high degree of forgetting (e.g., LwF on WHU-RS19 with FM = 0.5056). In contrast, our method effectively preserves historical knowledge through high-quality generative replay, reducing FM to 0.1571 and 0.2070 on UCMerced and WHU-RS19, respectively, thereby significantly alleviating forgetting. Nammour and SCN are methods specifically proposed for incremental learning in remote sensing scenes. Nammour constructs task-level feature prototypes for each historical task and attempts to mitigate forgetting by replaying these prototypes. However, in remote sensing scenes, each scene contains rich and diverse unlabeled semantic information, and a single feature prototype is insufficient to capture such complex and diverse feature distributions. SCN, on the other hand, employs two networks to perform mutual distillation and alternately fuses long- and short-term memories to alleviate forgetting. During training, however, differences in the networks' sensitivity to new tasks can lead to inconsistent gradient directions, causing historical knowledge to be overwritten or shifted. In addition, each distillation introduces approximation errors that accumulate as incremental tasks proceed, leading to distorted representations of historical tasks and overall limited CL performance.

Qualitative Analysis: Figs. 3 and 4 show the visual comparisons between real images and those generated by our method, providing an intuitive evaluation of the generative model's ability to preserve the semantic consistency. We selected several categories for visualization. In the UC Merced dataset, we chose beach, agriculture, and medium residential as examples, while in the WHU-RS19 dataset, we select parking, bridge, and beach. For each category, the first row displays real images, and the second row shows the corresponding images generated by the diffusion model. As shown in the figures, the generated images exhibit a high degree of similarity to the real images in terms of semantic content, successfully reproducing the characteristic features of different land-cover types. For instance, the generated images of the beach category clearly capture the shoreline and water structures, and the agriculture category retains the regular patterns of farmlands. These results demonstrate that our generative model can effectively capture the core semantic information of different scene categories, producing high-quality and semantically faithful replay samples that provide reliable support for reconstructing knowledge of previous tasks in CL.

C. Ablation Analysis

1) Performance Under Different Backbone: To further validate the generality and robustness of the proposed framework for class-incremental CL in remote sensing scene classification

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.



Fig. 3. Visualization of real and generated images for three categories (beach, agriculture, and medium residential) from the UC Merced dataset. For each category, the first row shows real images and the second row shows the corresponding generated images.

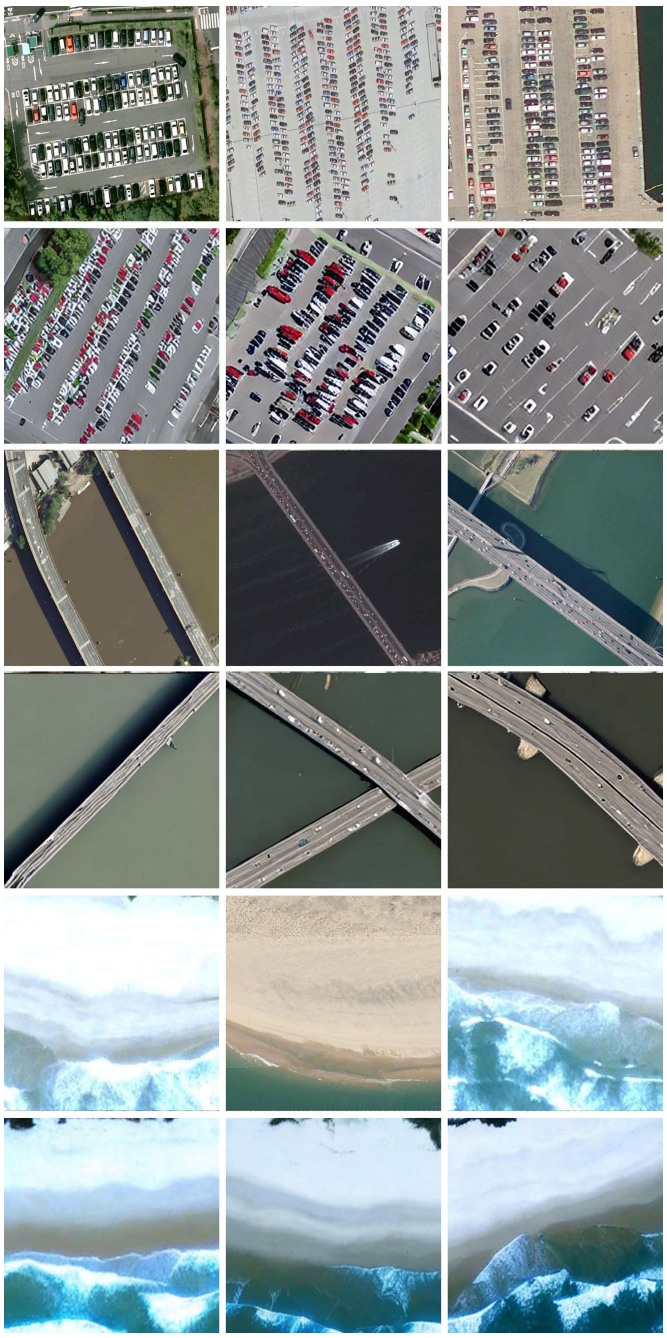


Fig. 4. Visualization of real and generated images for three categories (parking, bridge, and beach) from the WHU-RS dataset. For each category, the first row shows the real images and the second row shows the corresponding generated images.

tasks, we conducted experiments by replacing the backbone of the classification model with different architectures, including alexnet, ResNet-34, ResNet-50, DenseNet-121, and Vision Transformer (ViT). Under the same training and task-splitting settings, ablation experiments were performed on three datasets. We use the two-tail selection strategy for sample filtering. Specifically, for UC-Merced, AID, and WHU-RS19, we select 80 out of 90 samples, 250 out of 300 samples, and 50 out of 100 samples, respectively. The experimental results under this setup are shown in Table II. It can be observed that different backbones lead to significant performance variations under the same incremental configuration, indicating that the feature extraction capability of the backbone plays a crucial role in the proposed generative replay framework. In general, with stronger representation capacity, the model achieves higher AA and lower FM, demonstrating the robustness and adaptability of the proposed framework across different architectures.

Specifically, in terms of AA, deeper backbones, such as ResNet, DenseNet, and ViT consistently outperform AlexNet across all datasets, with improvements ranging from approximately 10% to 20% in some cases. Moreover, these

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.

TABLE II

PERFORMANCE OF OUR METHOD UNDER DIFFERENT BACKBONES. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD, AND THE SECOND-BEST RESULTS ARE INDICATED WITH AN UNDERLINE

	UC-Merced			AID			WHU-RS19		
Backbone	**AA↑**	**FM↓**	**IM↓**	**AA↑**	**FM↓**	**IM↓**	**AA↑**	**FM↓**	**IM↓**
Alexnet	0.7667	0.3071	-0.0572	0.6598	0.3791	**-0.0391**	0.7080	0.3893	-0.0345
Resnet-34	**0.8953**	**0.1571**	**-0.1143**	**0.7225**	**0.3199**	-0.0202	**0.8447**	**0.2070**	**-0.2141**
Resnet-50	0.8667	0.2000	-0.0143	0.6556	0.4002	-0.0218	0.8332	0.2224	-0.0417
Densenet-121	0.8476	0.2286	-0.0429	0.6139	0.4527	-0.0235	0.8046	0.2606	0.0000
Vit	0.8238	0.2643	-0.0143	0.6048	0.4560	-0.0383	0.7819	0.2980	0.0000

TABLE III

ABLATION RESULTS OF DIFFERENT GENERATED SAMPLE SELECTION STRATEGIES. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD, AND THE SECOND-BEST RESULTS ARE INDICATED WITH UNDERLINE

Selection Strategy	**num. of Samples**	**AA↑**	**FM↓**
No Selection	——	0.8347	0.2204
Topk	select 50 samples out of 60	0.8031	0.2626
	select 50 samples out of 53	0.8146	0.2471
	select 50 samples out of 51	0.8239	0.2348
	select 50 samples out of 100	0.8042	0.2610
	select 150 samples out of 300	**0.8472**	**0.1918**
	select 200 samples out of 300	0.7733	0.3023
Two-Tail	select 50 samples out of 60	0.8239	0.2348
	select 50 samples out of 100	0.8447	0.2070

models also exhibit lower FM values, suggesting better retention of previously learned knowledge and more stable incremental learning performance. Among ResNet, DenseNet, and ViT, the ResNet series performs best, where ResNet-34 and ResNet-50 achieve the highest and second-highest results, respectively. In comparison, DenseNet-121 and ViT show inferior performance to ResNet models. Taking the UC-Merced dataset as an example, ResNet-34 achieves an AA of 0.8953, whereas DenseNet-121 and ViT attain only 0.8476 and 0.8238, corresponding to decreases of approximately 5% and 7%, respectively. With respect to the forgetting metric, DenseNet-121 and ViT yield values of 0.2286 and 0.2643, both markedly higher than the 0.1571 of ResNet-34, suggesting substantially more severe forgetting. Similarly, on the AID dataset, the AA values of DenseNet-121 and ViT are lower than that of ResNet-34 by approximately 15% and 16%. On the WHU-RS19 dataset, DenseNet-121 and ViT also exhibit AA values that are approximately 4% and 7% lower than ResNet-34, and their FM values rise to 0.2606 and 0.2980, respectively, indicating a more pronounced degree of forgetting compared with the 0.2070 obtained by ResNet-34. In summary, ResNet-34 exhibits the most balanced and stable performance across different datasets, with a clear advantage in suppressing forgetting. Therefore, we adopt ResNet-34 as the default backbone in subsequent comparisons to fully exploit the semantic diversity of generative replay samples and to ensure a fair and reliable evaluation of the proposed framework.

2) Classifier-Based Sample Selection: Table III presents the performance of different selection strategies in the classifier's training stage, where the experiments are conducted on the WHU-RS19 dataset with Resnet-34 used as the backbone. No selection means that all generated samples are directly added into replay without any quality or confidence filtering. Top-k selection sorts generated samples by the classifier's prediction confidence and retains only the top-k highest confidence samples. In contrast, the proposed two-tail selection strategy selects samples from both ends of the confidence distribution, i.e., high-confidence head samples and relatively low confidence but still reliable tail samples.

The results show that, without selection, our method already achieves a competitive AA of 0.8347 with a relatively low FM of 0.2204. When we increase the number of generated samples per class and then filter 50 of them (the same quantity as the real samples per class), the top-k strategy unexpectedly leads to performance degradation. Regardless of selecting from 60, 53, or 51 samples, the AA drops by about 1%–3%, accompanied by increased FM. Moreover, the performance continues to deteriorate as the number of candidate samples grows. Visual inspection reveals that top-k filtering reduces diversity as k increases: the classifier tends to favor visually similar samples, resulting in highly redundant high-confidence images. Such overconfident but homogeneous data significantly weakens the coverage and representativeness of the replay distribution, ultimately harming CL performance. Further analysis shows that due to the strong generative ability of SD, the classifier correctly predicts nearly all generated samples with confidence values above 0.9. This indicates that semantic consistency of generated images is well guaranteed, suggesting that diversity should be preserved rather than aggressively filtered out. Under this observation, our two-tail method performs markedly better than top-k in both the "select 50 samples out of 60" and "select 50 samples out of 100" settings, yielding 2%–5% higher AA while reducing forgetting by 10%–20%. This demonstrates that two-tail effectively introduces semantically diverse samples with larger distributional variance, thereby improving replay coverage. However, we note that even with two-tail, the "select 50 samples out of 60" configuration is still inferior to no selection. When the candidate pool increases to 100, two-tail starts to show significant advantages, implying that an adequately large candidate space is crucial for mining meaningful diversity.

Finally, we further investigate the effect of replay sample quantity under the top-k strategy. Interestingly, increasing the number of replay samples does not necessarily improve the performance. For instance, "select 100 samples out of 200" results in worse accuracy than "select 50 samples out of 60." Only when the replay pool grows large enough to cover broader historical variation can the performance be substantially improved. Based on the above observations, we conclude that two-tail with a candidate size of 100 (select two-tail 100)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
STORAGE SPACE COMPARISON. NOTE THAT FOR ICARL, WHICH STORES EXEMPLAR SAMPLES, WE RUN THE METHOD THREE TIMES AND REPORT THE AVERAGE STORAGE CONSUMPTION

dataset	iCaRL [21]	ours
UC-Merced	55.85MB	12.24MB
AID	116.58MB	30.6MB
WHU-RS19	28.54MB	18.36MB

achieves a desirable tradeoff between diversity enhancement and redundancy suppression. Therefore, we adopt select two-tail 100 as the default generative replay strategy throughout the comparative experiments, ensuring representative coverage of generated samples while maintaining the efficiency and system stability.

D. Storage Consumption Analysis

To verify our method's storage advantage, we compare its required storage to iCaRL [21], which stores old samples. We conduct this comparison under three CL settings on the same datasets. Table IV reports the storage size for our method and iCaRL after the final task. In iCaRL, 20 samples per class are stored. Storing more samples increases space requirements. For the UC-Merced dataset, before the final task, we store 280 samples from the first two tasks, which takes 55.85 MB. For the AID dataset, 500 samples from the first five tasks occupy 116.58 MB. For the WHU-RS19 dataset, 280 samples from the first four tasks take up to 28.54 MB. Our method uses far less storage on all datasets than storing real data.

VI. CONCLUSION

This article proposes an SD-based generative replay framework for CL in remote sensing scene classification. Using the LoRA fine-tuning strategy, SD can quickly adapt to changing task distributions and generate representative samples from previous tasks for replay. This approach effectively reduces the catastrophic forgetting without storing real data and greatly lowers storage costs compared with traditional experience replay methods. In addition, a confidence-based data selection mechanism is introduced to ensure the quality of generated samples, further enhancing knowledge retention. Experiments on multiple remote sensing datasets show that the proposed method achieves better performance than existing CL approaches.

REFERENCES

[1] G. Cheng, X. Xie, J. Han, L. Guo, and G.-S. Xia, "Remote sensing image scene classification meets deep learning: Challenges, methods, benchmarks, and opportunities," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 3735–3756, 2020.

[2] N. Longbotham, C. Chaapel, L. Bleiler, C. Padwick, W. J. Emery, and F. Pacifici, "Very high resolution multiangle urban classification analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 4, pp. 1155–1170, Apr. 2012.

[3] T. R. Martha, N. Kerle, C. J. van Westen, V. Jetten, and K. V. Kumar, "Segment optimization and data-driven thresholding for knowledge-based landslide detection by object-based image analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 12, pp. 4928–4943, Dec. 2011.

[4] Y. Li, Y. Zhang, X. Huang, and A. L. Yuille, "Deep networks under scene-level supervision for multi-class geospatial object detection from remote sensing images," *ISPRS J. Photogramm. Remote Sens.*, vol. 146, pp. 182–196, Dec. 2018.

[5] H. Alhichri, "Multitask classification of remote sensing scenes using deep neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2018, pp. 1195–1198.

[6] B. Xi, W. Zhang, J. Li, R. Song, and Y. Li, "HyperTaFOR: Task-adaptive few-shot open-set recognition with spatial–spectral selective transformer for hyperspectral imagery," *IEEE Trans. Image Process.*, vol. 34, pp. 4148–4160, 2025.

[7] B. Xi, W. Zhang, J. Li, R. Song, and Y. Li, "HyperCASR: Spectral–spatial open-set recognition with category-aware semantic reconstruction for hyperspectral imagery," *IEEE Trans. Image Process.*, vol. 34, pp. 7642–7655, 2025.

[8] B. Xi et al., "HyLiOSR: Staged progressive learning for joint open-set recognition of hyperspectral and LiDAR data," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5506714.

[9] H. Li et al., "CLRS: Continual learning benchmark for remote sensing image scene classification," *Sensors*, vol. 20, no. 4, p. 1226, Feb. 2020.

[10] L. Wang, X. Zhang, H. Su, and J. Zhu, "A comprehensive survey of continual learning: Theory, method and application," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5362–5383, Aug. 2024.

[11] X. Lu, X. Sun, W. Diao, Y. Feng, P. Wang, and K. Fu, "LIL: Lightweight incremental learning approach through feature transfer for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5611320.

[12] N. Ammour, Y. Bazi, H. Alhichri, and N. Alajlan, "Continual learning approach for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[13] N. Ammour, "Continual learning using data regeneration for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[14] S. D. Bhat, B. Banerjee, S. Chaudhuri, and A. Bhattacharya, "CILEA-NET: Curriculum-based incremental learning framework for remote sensing image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 5879–5890, 2021.

[15] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci. USA*, vol. 114, no. 13, pp. 3521–3526, 2017.

[16] H. Ritter, A. Botev, and D. Barber, "Online structured Laplace approximations for overcoming catastrophic forgetting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 3738–3748.

[17] J. Schwarz et al., "Progress & compress: A scalable framework for continual learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4528–4537.

[18] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3987–3995.

[19] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. Torr, "Riemannian walk for incremental learning: Understanding forgetting and intransigence," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 532–547.

[20] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2935–2947, Dec. 2018.

[21] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "ICaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2001–2010.

[22] P. Dhar, R. V. Singh, K. Peng, Z. Wu, and R. Chellappa, "Learning without memorizing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5138–5146.

[23] P. Buzzega, M. Boschini, A. Porrello, D. Abati, and S. Calderara, "Dark experience for general continual learning: A strong, simple baseline," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 15920–15930.

[24] H. Shin, J. K. Lee, J. Kim, and J. Kim, "Continual learning with deep generative replay," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 2994–3003.

[25] C. Wu, L. Herranz, X. Liu, Y. Wang, J. Van De Weijer, and B. Raducanu, "Memory replay GANs: Learning to generate new categories without forgetting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 5962–5972.

[26] X. Liu et al., "Generative feature replay for class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, Jun. 2020, pp. 226–227.

[27] G. M. van de Ven, H. T. Siegelmann, and A. S. Tolias, "Brain-inspired replay for continual learning with artificial neural networks," *Nature Commun.*, vol. 11, no. 1, p. 4069, Aug. 2020.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.

[28] G. Zeng, Y. Chen, B. Cui, and S. Yu, "Continual learning of context-dependent processing in neural networks," *Nature Mach. Intell.*, vol. 1, no. 8, pp. 364–372, Aug. 2019.
[29] Y. Guo, W. Hu, D. Zhao, and B. Liu, "Adaptive orthogonal projection for batch and online continual learning," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 6, pp. 6783–6791.
[30] H. Liu and H. Liu, "Continual learning with recursive gradient optimization," 2022, *arXiv:2201.12522*.
[31] K. Javed and M. White, "Meta-learning representations for continual learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1818–1828.
[32] S. Beaulieu et al., "Learning to continually learn," 2020, *arXiv:2002.09571*.
[33] A. A. Rusu et al., "Progressive neural networks," 2016, *arXiv:1606.04671*.
[34] C. Fernando et al., "PathNet: Evolution channels gradient descent in super neural networks," 2017, *arXiv:1701.08734*.
[35] R. Ramesh and P. Chaudhari, "Model zoo: A growing 'brain' that learns continually," 2021, *arXiv:2106.03027*.
[36] L. Wang, X. Zhang, Q. Li, J. Zhu, and Y. Zhong, "CoSCL: Cooperation of small continual learners is stronger than a big one," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 254–271.
[37] J. Yoon, E. Yang, J. Lee, and S. J. Hwang, "Lifelong learning with dynamically expandable networks," 2017, *arXiv:1708.01547*.
[38] A. Mallya and S. Lazebnik, "PackNet: Adding multiple tasks to a single network by iterative pruning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7765–7773.
[39] S. V. Mehta, D. Patil, S. Chandar, and E. Strubell, "An empirical investigation of the role of pre-training in lifelong learning," *J. Mach. Learn. Res.*, vol. 24, no. 214, pp. 1–50, 2021.
[40] G. Shi, Y. Wu, J. Liu, S. Wan, W. Wang, and T. Lu, "Incremental few-shot semantic segmentation via embedding adaptive-update and hyper-class representation," in *Proc. 30th ACM Int. Conf. Multimedia*, Oct. 2022, pp. 5547–5556.
[41] E. Fini, V. G. T. Da Costa, X. Alameda-Pineda, E. Ricci, K. Alahari, and J. Mairal, "Self-supervised models are continual learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 9621–9630.
[42] H. Cha, J. Lee, and J. Shin, "Co2L: Contrastive continual learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 9496–9505.
[43] Q. Pham, L. Chenghao, and S. C. H. Hoi, "DualNet: Continual learning, fast and slow," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 16131–16144.
[44] J. E. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, Oct. 2021, p. 3.
[45] A. Chaudhry et al., "On tiny episodic memories in continual learning," 2019, *arXiv:1902.10486*.
[46] M. Riemer et al., "Learning to learn without forgetting by maximizing transfer and minimizing interference," 2018, *arXiv:1810.11910*.
[47] G. Gupta, K. Yadav, and L. Paull, "Look-ahead meta learning for continual learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 11588–11598.
[48] D. Ye, J. Peng, H. Li, and L. Bruzzone, "Better memorization, better recall: A lifelong learning framework for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5626814.
[49] S. Khanna et al., "DiffusionSat: A generative foundation model for satellite imagery," 2023, *arXiv:2312.03606*.
[50] D. Tang, X. Cao, X. Hou, Z. Jiang, J. Liu, and D. Meng, "CRS-diff: Controllable remote sensing image generation with diffusion model," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5638714.
[51] D. Sun, Y. Bao, J. Liu, and X. Cao, "A lightweight sparse focus transformer for remote sensing image change captioning," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 18727–18738, 2024.
[52] Y. Xiao, Q. Yuan, K. Jiang, J. He, X. Jin, and L. Zhang, "EDiffSR: An efficient diffusion probabilistic model for remote sensing image super-resolution," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5601514.
[53] X. Rui, X. Cao, L. Pang, Z. Zhu, Z. Yue, and D. Meng, "Unsupervised hyperspectral pansharpening via low-rank diffusion model," *Inf. Fusion*, vol. 107, Jul. 2024, Art. no. 102325.
[54] Y. Liu, J. Yue, S. Xia, P. Ghamisi, W. Xie, and L. Fang, "Diffusion models meet remote sensing: Principles, methods, and perspectives," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 4708322.
[55] C. Schuhmann et al., "LAION-5B: An open large-scale dataset for training next generation image-text models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 25278–25294.
[56] Y. Yang and S. Newsam, "Bag-of-visual-words and spatial extensions for land-use classification," in *Proc. 18th SIGSPATIAL Int. Conf. Adv. Geograph. Inf. Syst.*, Nov. 2010, pp. 270–279.
[57] G.-S. Xia et al., "AID: A benchmark data set for performance evaluation of aerial scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3965–3981, Jul. 2017.
[58] D. Dai and W. Yang, "Satellite image classification via two-layer sparse coding with biased image representation," *IEEE Geosci. Remote Sens. Lett.*, vol. 8, no. 1, pp. 173–176, Jan. 2011.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:10:41 UTC from IEEE Xplore. Restrictions apply.